第四章 神经网络的学习

本章的核心目标是：

- 理解神经网络中的“学习”到底指什么
- 明白训练集、测试集与泛化能力的概念
- 掌握损失函数的作用
- 理解数值微分、偏导数和梯度
- 学会梯度下降的基本思想
- 理解神经网络学习本质上是在优化参数

4.1 从数据中学习

前几章中，我们已经知道：

- 神经网络可以通过多层结构表达复杂函数
- 每一层中都有权重和偏置

但问题在于：

> 这些参数不可能靠人手工一个个设置。

因此，神经网络真正重要的地方在于：

> 让模型从数据中自动学习参数。

所谓“学习”，就是让参数逐渐适应数据，使模型输出越来越接近正确答案。

4.2 训练数据与测试数据

机器学习中，通常会把数据划分为：

- 训练数据（training data）
- 测试数据（test data）

训练数据用于：

- 调整模型参数
- 让网络从中学习规律

测试数据用于：

- 检查模型对新数据的表现
- 评估模型是否具备泛化能力

这里最关键的是：

模型不能只会“背训练集答案”，而必须对没见过的数据也表现良好。

这就是泛化能力的意义。

4.3 损失函数

为了让模型学习，必须先定义一个标准来衡量“当前预测有多差”。
这个标准就是：

损失函数（loss function）

损失函数的作用是：

- 把预测结果的好坏映射成一个数值
- 损失越小，说明预测越好
- 训练的目标就是让损失尽可能小

4.3.1 均方误差

均方误差（Mean Squared Error）定义为：

$$E = \frac{1}{2}\sum_k (y_k - t_k)^2$$

其中：

- y_k 是模型输出
- t_k 是真实标签

代码实现：

```
import numpy as np

def mean_squared_error(y, t):
    return 0.5 * np.sum((y - t) ** 2)
```

均方误差的含义非常直观：

预测值和真实值差得越大，损失就越大。

4.3.2 交叉熵误差

分类问题中更常用的是交叉熵误差：

$$E = -\sum_k t_k \log y_k$$

代码实现：

```
import numpy as np

def cross_entropy_error(y, t):
    delta = 1e-7
    return -np.sum(t * np.log(y + delta))
```

其中加入一个很小的 delta，是为了防止出现 log(0)。

交叉熵的一个重要性质是：

- 如果正确类别概率高，损失就小
- 如果正确类别概率低，损失就大

因此它特别适合分类任务。

4.3.3 mini-batch 学习

在实际训练中，数据量往往很大，不可能每次都用全部数据来更新参数。

因此常见策略是：

> 每次只随机抽取一小批数据进行训练。

这就是 mini-batch。

示例代码：

```
import numpy as np

train_size = 60000
batch_size = 100

batch_mask = np.random.choice(train_size, batch_size)
```

mini-batch 的优点：

- 计算更高效
- 更新更灵活
- 更适合大规模训练

4.4 为什么不能直接用精度当损失函数

表面上看，分类任务最终关心的是识别精度。但训练时通常不直接优化精度，而是优化损失函数。

原因在于：

> 精度对参数的小变化往往不敏感，不适合作为连续优化目标。

例如：

- 参数稍微变化一点

- 预测类别可能完全不变
- 精度就不会发生变化

这样就很难知道参数应该朝哪个方向调整。
而损失函数通常是平滑变化的，因此更适合作为优化对象。

4.5 数值微分

神经网络学习的核心问题是：

> 怎样调整参数，才能让损失函数变小？

这就需要微分。
微分的直观意义是：

> 某一点附近，函数变化得有多快。

4.5.1 导数的近似

一个最简单的近似写法是：

```
def numerical_diff_bad(f, x):
    h = 1e-50
    return (f(x + h) - f(x)) / h
```

但这种写法不稳定，原因是：

- h 太小会带来浮点误差
- 前向差分的精度较差

更好的写法是中心差分：

```
def numerical_diff(f, x):
    h = 1e-4
    return (f(x + h) - f(x - h)) / (2 * h)
```

4.5.2 示例函数

例如：

$$f(x) = 0.01x^2 + 0.1x$$

代码：

```
def function_1(x):
    return 0.01 * x**2 + 0.1 * x
```

数值微分可以帮助我们观察函数在某一点的斜率。

4.6 偏导数

如果函数有多个变量，就需要使用偏导数。
例如：

$$f(x_0, x_1) = x_0^2 + x_1^2$$

偏导数的含义是：

- 对 x_0 求导时，把 x_1 看作常数
- 对 x_1 求导时，把 x_0 看作常数

代码形式：

```
def function_2(x):
    return x[0]**2 + x[1]**2
```

偏导数描述的是：

> 沿某一个方向变化时，函数值如何变化。

4.7 梯度

对于多变量函数，把所有偏导数组合起来，就得到了梯度。
例如：

$$f(x_0, x_1) = x_0^2 + x_1^2$$

其梯度可写成：

$$\left(\frac{\partial f}{\partial x_0}, \frac{\partial f}{\partial x_1}\right)$$

梯度的直观含义是：

> 函数增长最快的方向。

因此，若想让函数值变小，就应沿着负梯度方向移动。

4.7.1 求梯度的代码

```
import numpy as np

def numerical_gradient(f, x):
    h = 1e-4
    grad = np.zeros_like(x)

    for idx in range(x.size):
        tmp_val = x[idx]
```

```
        x[idx] = tmp_val + h
        fxh1 = f(x)

        x[idx] = tmp_val - h
        fxh2 = f(x)

        grad[idx] = (fxh1 - fxh2) / (2 * h)
        x[idx] = tmp_val

    return grad
```

4.8 梯度下降法

既然梯度表示函数上升最快的方向，那么为了让函数值下降，就应该沿着梯度反方向更新参数。这就是梯度下降法：

$$x = x - \eta \nabla f(x)$$

其中：

- x 是参数
- η 是学习率
- $\nabla f(x)$ 是梯度

学习率决定更新步长：

- 太大：容易越过最优点
- 太小：收敛速度很慢

代码实现：

```
def gradient_descent(f, init_x, lr=0.01, step_num=100):
    x = init_x

    for i in range(step_num):
        grad = numerical_gradient(f, x)
        x -= lr * grad

    return x
```

4.9 神经网络中的梯度

在神经网络中，参数包括：

- 权重 W
- 偏置 b

而学习的本质就是：

找一组参数，使损失函数尽可能小。

因此，神经网络训练本质上就是一个优化问题：

- 先定义损失函数
- 再对参数求梯度
- 最后用梯度下降法更新参数

4.9.1 一个简单网络示例

```
import numpy as np

class simpleNet:
    def __init__(self):
        self.W = np.random.randn(2, 3)

    def predict(self, x):
        return np.dot(x, self.W)

    def loss(self, x, t):
        z = self.predict(x)
        y = softmax(z)
        loss = cross_entropy_error(y, t)
        return loss
```

这里：

- `W` 是参数
- `predict` 做前向计算
- `loss` 用于计算损失

4.9.2 对参数求梯度

```
f = lambda w: net.loss(x, t)
dW = numerical_gradient(f, net.W)
```

这里的 `dW` 就表示损失函数对参数 W 的梯度。

4.10 学习步骤总结

神经网络学习通常可以概括为四步：

1. 从训练数据中随机取 mini-batch
2. 计算损失函数关于参数的梯度
3. 沿梯度反方向更新参数
4. 重复上述步骤直到收敛

这就是后面所有深度学习训练流程的基本框架。

4.11 本章小结

第四章的核心是:

- 神经网络不是手工调参数，而是从数据中学习参数
- 学习依赖损失函数来衡量预测好坏
- 数值微分、偏导数、梯度给出了参数更新方向
- 梯度下降法给出了具体更新规则

需要重点掌握:

- 均方误差
- 交叉熵误差
- mini-batch
- 数值微分
- 梯度
- 梯度下降

本章是神经网络“会计算”向“会学习”的过渡。下一章将进一步讨论如何更高效地计算梯度，也就是误差反向传播法。

4.12 附：本章核心代码

```
import numpy as np

def mean_squared_error(y, t):
    return 0.5 * np.sum((y - t) ** 2)

def cross_entropy_error(y, t):
    delta = 1e-7
    return -np.sum(t * np.log(y + delta))

def numerical_diff(f, x):
    h = 1e-4
    return (f(x + h) - f(x - h)) / (2 * h)

def numerical_gradient(f, x):
    h = 1e-4
    grad = np.zeros_like(x)

    for idx in range(x.size):
        tmp_val = x[idx]

        x[idx] = tmp_val + h
        fxh1 = f(x)

        x[idx] = tmp_val - h
        fxh2 = f(x)
```

```
        grad[idx] = (fxh1 - fxh2) / (2 * h)
        x[idx] = tmp_val

    return grad

def gradient_descent(f, init_x, lr=0.01, step_num=100):
    x = init_x

    for i in range(step_num):
        grad = numerical_gradient(f, x)
        x -= lr * grad

    return x
```